

RotateBox Widget Documentation

Overview

`RotateBox` is an advanced Flutter animation widget that demonstrates how to use **AnimationController** with **AnimatedBuilder** for high-performance animations.

This widget continuously rotates an icon without calling `setState()` on every frame, making it efficient and smooth.

Why This Widget Is Advanced

- Uses `SingleTickerProviderStateMixin`
 - Uses `AnimationController` directly
 - Uses `AnimatedBuilder` for optimized rebuilds
 - Avoids unnecessary widget rebuilds
-

Full Source Code

```
class RotateBox extends StatefulWidget {  
  @override  
  _RotateBoxState createState() => _RotateBoxState();  
}  
  
class _RotateBoxState extends State<RotateBox>  
  with SingleTickerProviderStateMixin {  
  
  late AnimationController controller;  
  
  @override  
  void initState() {  
    super.initState();  
    controller = AnimationController(  
      vsync: this,  
      duration: Duration(seconds: 2),  
    )..repeat();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return AnimatedBuilder(  

```

```

        animation: controller,
        child: Icon(Icons.refresh, size: 50),
        builder: (context, child) {
            return Transform.rotate(
                angle: controller.value * 6.28,
                child: child,
            );
        },
    );
}

@override
void dispose() {
    controller.dispose();
    super.dispose();
}
}

```

Code Explanation (Step by Step)

1. SingleTickerProviderStateMixin

```
with SingleTickerProviderStateMixin
```

- Provides a `Ticker` for animations - Required for `AnimationController` - Uses fewer resources than multiple tickers

2. AnimationController

```

controller = AnimationController(
    vsync: this,
    duration: Duration(seconds: 2),
)..repeat();

```

- Controls animation timing - `repeat()` makes the animation infinite - Value range: **0.0 → 1.0**

3. AnimatedBuilder

```

AnimatedBuilder(
  animation: controller,
  child: Icon(Icons.refresh),
  builder: (context, child) {...}
)

```

- Listens to animation changes - Rebuilds **only the animated part** - `child` is cached (performance boost)

4. `Transform.rotate`

```
angle: controller.value * 6.28
```

- `6.28` $\approx 2\pi$ radians (360°) - Converts animation value into rotation angle

Performance Benefits

Feature	Benefit
AnimatedBuilder	Avoids full rebuild
Cached child	Faster rendering
No setState	Smooth animation

When to Use This Pattern

- Loading spinners
- Rotating icons
- Custom animations
- Dashboard indicators

Common Mistakes

✗ Forgetting `dispose()`
✗ Using `setState()` for animations
 ✗ Rebuilding heavy widgets inside animation

Customization Examples

Change Speed

```
duration: Duration(milliseconds: 800)
```

Rotate Any Widget

```
child: Image.asset('logo.png')
```

Related Advanced Widgets

- AnimatedWidget
- TweenAnimationBuilder
- ImplicitlyAnimatedWidget
- CustomPainter

Summary

`RotateBox` is a **production-ready animation pattern** in Flutter. Mastering this means you can create smooth, efficient animations without hurting app performance.

You can freely edit this document.  You can export it later as **PDF / DOCX** if needed.